

MODULE 4 LAB WORKBOOK

Robot Framework Test Automation

Name – Sristi Saha

Institute – Institute of Engineering & Management, Kolkata

Enrollment No. – 12023052016066

Module	Robot Framework Test Automation
Automation framework	Robot Framework
Browser automation	SeleniumLibrary and Google Chrome
API automation	RequestsLibrary and JSONPlaceholder
Custom extensions	Python-based Robot Framework libraries
Execution and reporting	Tags, Pabot, output.xml, log.html, report.html

Experiment List

Experiment	Experiment Title	Primary Concepts
1	Basic Syntax and Keywords	Browser navigation, Input Text, element assertions
2	Variables and Data-Driven Automation	Variables, CSV test data, repeated execution
3	Custom Keywords and Libraries	Python library, BuiltIn string and math operations
4	Assertions and Verification	UI assertions and API response validation
5	Test Setup and Teardown	Reusable login setup and cleanup
6	Tags and Test Execution	Tags, include/exclude, parallel execution
7	Reports and Logs	HTML reports, XML output, custom logging

Experiment 1: Basic Syntax and Keywords

Problem Statement

Create a Robot Framework test case that opens a browser, navigates to a specified URL, enters text into form fields, and verifies that a required element and success message are present.

Objective

To practice Robot Framework test syntax and common SeleniumLibrary keywords for browser control, form entry, and page verification.

Tools, Software, and Concepts Used

- Python 3
- Robot Framework and SeleniumLibrary
- Google Chrome and Selenium WebDriver
- Keywords: Open Browser, Input Text, Page Should Contain Element, Page Should Contain

Implementation Steps

1. Create an assignment folder and install Robot Framework and SeleniumLibrary.
2. Create assignment1.robot with Settings, Variables, and Test Cases sections.
3. Open the login page, enter the sample credentials, and verify the login form element.
4. Submit the form and verify the secure-area confirmation.

Source Code —

assignment1.robot

***** Settings *****

Library SeleniumLibrary

***** Variables *****

\${URL} https://the-internet.herokuapp.com/login

\${BROWSER} Chrome

***** Test Cases *****

Open Browser And Fill Login Form

Open Browser \${URL} \${BROWSER}

Maximize Browser Window

Sleep 2s

Verify the username field exists

Page Should Contain Element id=username

Enter username and password

Input Text id=username tomsmith

Input Text id=password SuperSecretPassword!

Pause to take screenshot

Sleep 10s

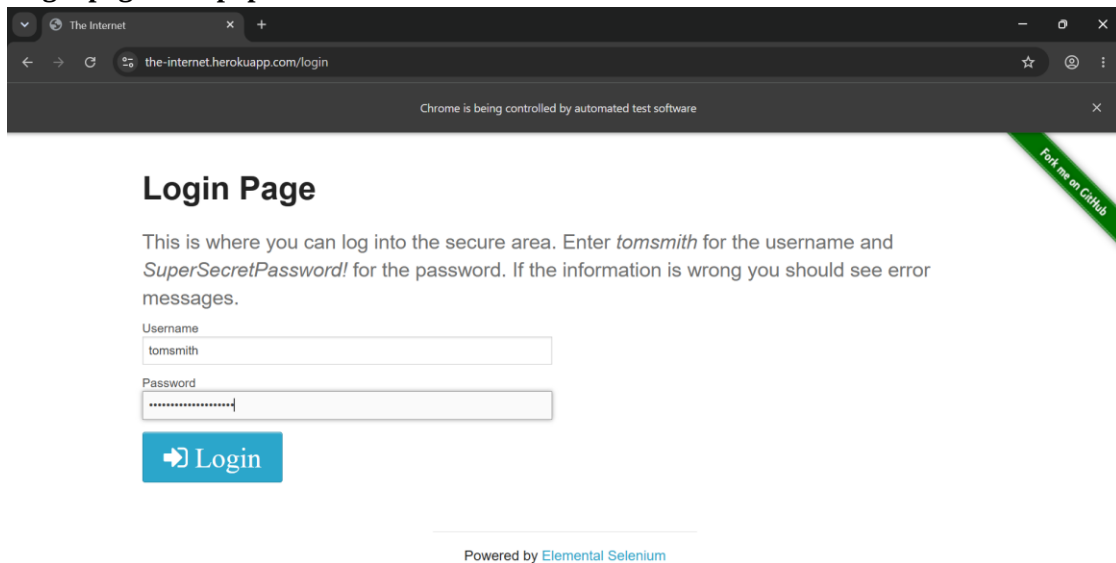
```
# Submit the form
Click Button    css=button[type='submit']
Sleep    3s

# Verify successful login
Page Should Contain    You logged into a secure area!

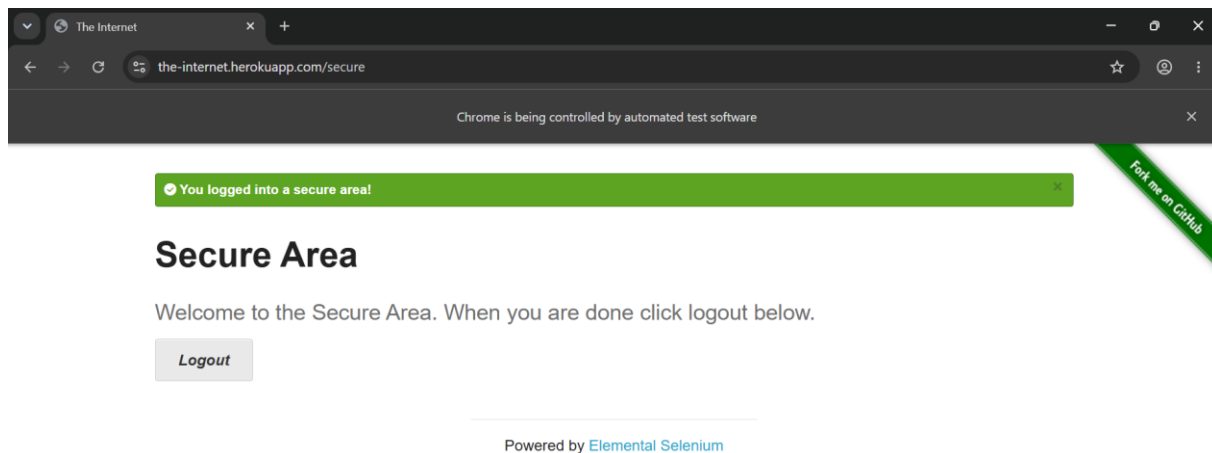
# Pause to capture the result
Sleep    10s
Close Browser
```

Output

1.1 Login page with populated fields



1.2 Successful login page



1.3 Terminal execution result

```
Terminal Local x Local (2) x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment1> robot assignment1.robot
=====
Assignment1
=====
Open Browser And Fill Login Form
DevTools listening on ws://127.0.0.1:52453/devtools/browser/45842126-7a9a-4977-b825-7a7d23f557f5
..[10148:24956:0928/002840.340:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[10148:24956:0928/002840.375:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[10148:24956:0928/002840.375:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
....[10148:4544:0928/002847.545:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 28116: Access is denied. (0x5)
[10148:37884:0928/002849.790:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 25860: Access is denied. (0x5)
[10148:4544:0928/002849.792:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 25860: Access is denied. (0x5)
[10148:37884:0928/002849.953:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 39724: Access is denied. (0x5)
[10148:4544:0928/002852.179:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 34728: Access is denied. (0x5)
..Created TensorFlow Lite XNNPACK delegate for CPU.
[10148:37884:0928/002854.330:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 39672: Access is denied. (0x5)
Open Browser And Fill Login Form ..[10148:24956:0928/002903.345:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Open Browser And Fill Login Form | PASS |
=====
Assignment1 | PASS |
=====
1 test, 1 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment1\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment1\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment1\report.html
```

Result and Observation

The test automates the login form and checks the expected success message. The browser page and terminal output provide execution evidence. The test demonstrates the Robot Framework file structure and browser automation keywords. The explicit element wait helps synchronize the test with page loading.

Conclusion

The experiment introduces basic Robot Framework syntax, SeleniumLibrary browser operations, text entry, and page-content assertions.

Experiment 2: Variables and Data-Driven Automation

Problem Statement

Create a login test that stores test values in variables and reads multiple credential combinations from an external CSV file, repeating the same browser workflow and checking each expected outcome.

Objective

To demonstrate Robot Framework variables, looping, external CSV data, and data-driven execution using a reusable Python CSV reader.

Tools, Software, and Concepts Used

- Python 3, Robot Framework, SeleniumLibrary
- Custom Python CSVLibrary.py and testdata.csv
- Variables, FOR loop, conditional IF/ELSE, CSV parsing, assertions

Implementation Steps

1. Create assignment2.robot, CSVLibrary.py, and testdata.csv in the same assignment folder.
2. Add username, password, and expected-result columns to the CSV file.
3. Import SeleniumLibrary and the custom CSV library in the Robot settings.
4. Read the CSV rows and loop through each credential set.

Source Code —

assignment2.robot

***** Settings *****

Library SeleniumLibrary

Library CSVlibrary.py

***** Variables *****

\${URL} https://the-internet.herokuapp.com/login

\${BROWSER} Chrome

\${DATA_FILE} testdata.csv

***** Test Cases *****

Data Driven Login Test

\${test_data}= read_test_data *\${DATA_FILE}*

FOR *\${row}* IN *@{test_data}*

\${username}= Set Variable *\${row}[0]*

\${password}= Set Variable *\${row}[1]*

\${expected}= Set Variable *\${row}[2]*

Open Browser *\${URL}* *\${BROWSER}*

Maximize Browser Window

Input Text id=username *\${username}*

```

Input Text  id=password  ${password}
Sleep  5s

Click Button  css=button[type='submit']
Sleep  2s

IF  '${expected}' == 'success'
    Page Should Contain  You logged into a secure area!
ELSE
    ${actual}=  Get Text  id=flash
    Should Contain  ${actual}  invalid
END
    Sleep  5s
Close Browser
END

```

CSVlibrary.py

```

import csv
class CSVlibrary:
    def read_test_data(self, file_path):
        """Read test cases from an external CSV file."""
        with open(file_path, mode="r", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            return [
                [row["username"], row["password"], row["expected"]]
                for row in reader
            ]

```

testdata.csv

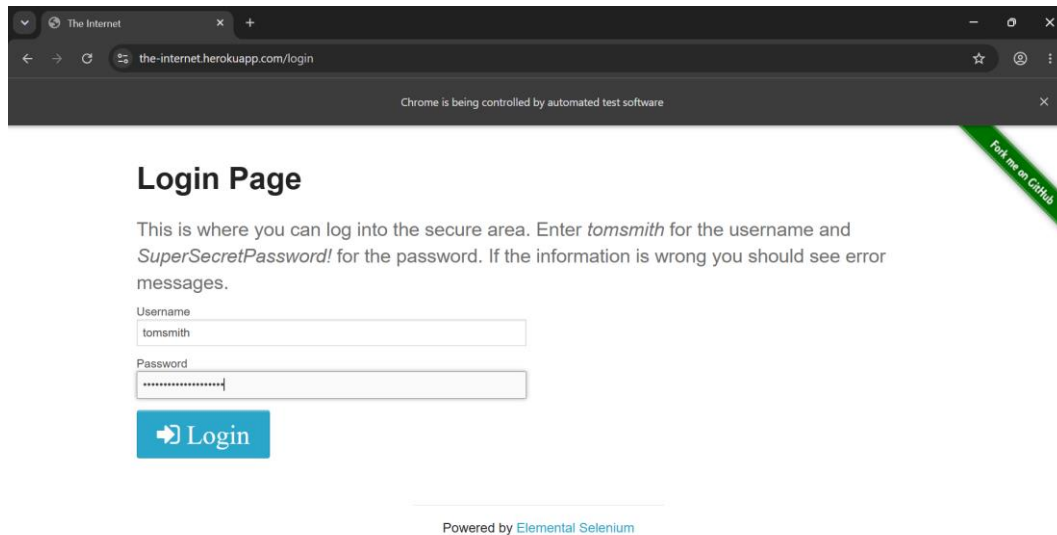
```

username,password,expected
tomsmith,SuperSecretPassword!,success
invaliduser,wrongpassword,failure
tomsmith,wrongpassword,failure

```

Output

2.1 Login form for a data row



Chrome is being controlled by automated test software

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

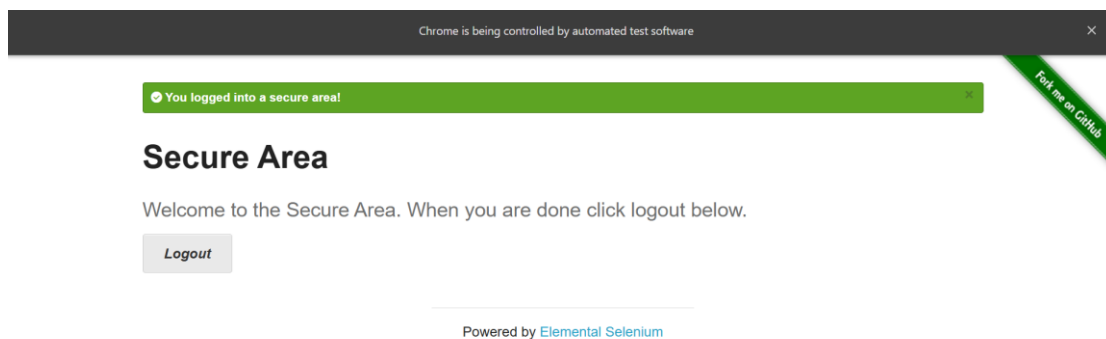
Username
tomsmith

Password
SuperSecretPassword!

Login

Powered by [Elemental Selenium](#)

2.2 Successful and unsuccessful outcomes



Chrome is being controlled by automated test software

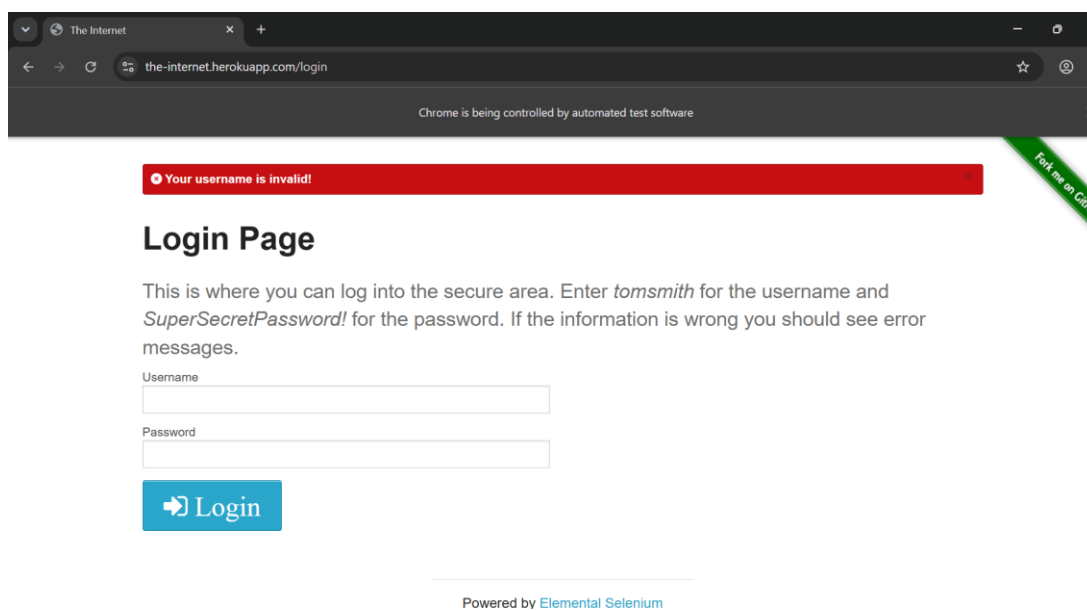
You logged into a secure area!

Secure Area

Welcome to the Secure Area. When you are done click logout below.

Logout

Powered by [Elemental Selenium](#)



Chrome is being controlled by automated test software

Your username is invalid!

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password
SuperSecretPassword!

Login

Powered by [Elemental Selenium](#)

2.3 Terminal execution result

```
Terminal Local x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment2> robot assignment2.robot
=====
Assignment2
=====
Data Driven Login Test
DevTools listening on ws://127.0.0.1:60218/devtools/browser/10ffc1c2-f0fd-4184-907b-6ca5afe2c47e
[31124:39692:0928/004652.136:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Created TensorFlow Lite XNNPACK delegate for CPU.
[31124:33208:0928/004700.611:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 39176: Access is denied. (0x5)
[31124:17496:0928/004702.743:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 23204: Access is denied. (0x5)

DevTools listening on ws://127.0.0.1:60103/devtools/browser/9540843d-f97e-4081-8331-b4285d504c55
[34104:40328:0928/004714.061:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT

DevTools listening on ws://127.0.0.1:55192/devtools/browser/348d6023-7426-486b-8e4f-1d6081ed3e1f
[25860:14928:0928/004737.912:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Created TensorFlow Lite XNNPACK delegate for CPU.
Data Driven Login Test | PASS |
-----
Assignment2 | PASS |
1 test, 1 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment2\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment2\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment2\report.html
```

Result and Observation

The test reads three credential rows and executes the same login flow for each. The successful row checks the secure-area message, while invalid rows check for an invalid-credentials alert. The expected column uses the labels success and failure. The invalid-case assertion checks the actual alert text for “invalid,” rather than expecting the literal word “failure.” Keep all three files in the same folder so the relative CSV and library paths resolve.

Conclusion

The experiment demonstrates variable-driven and data-driven browser testing with external CSV input and reusable test logic.

Experiment 3: Custom Keywords and Libraries

Problem Statement

Create a custom Robot Framework keyword in Python to calculate the sum of two numbers, and use the BuiltIn library to perform mathematical and string operations within a test case.

Objective

To extend Robot Framework with a Python library and use BuiltIn keywords for arithmetic, string manipulation, length calculation, logging, and assertions.

Tools, Software, and Concepts Used

- Python 3 and Robot Framework
- MathLibrary.py custom library
- BuiltIn library: Evaluate, Get Length, Log To Console
- Numeric and string comparison assertions

Implementation Steps

1. Create MathLibrary.py with a MathLibrary class and calculate_sum method.
2. Import MathLibrary.py and BuiltIn in assignment3.robot.
3. Call Calculate Sum with two numeric variables and verify the result.
4. Use Evaluate for multiplication and uppercase conversion, and Get Length for string length.
5. Log the values and inspect the test result.

Source Code —

assignment3.robot

```
*** Settings ***
Library    MathLibrary.py
Library    BuiltIn

*** Variables ***
${FIRST_NUMBER}    25
${SECOND_NUMBER}    35
${TEXT}            robot framework automation

*** Test Cases ***
Custom Keyword And BuiltIn Operations
    ${result}=    Calculate Sum    ${FIRST_NUMBER}    ${SECOND_NUMBER}
    Should Be Equal As Numbers    ${result}    60

    ${product}=    Evaluate    ${FIRST_NUMBER} * ${SECOND_NUMBER}
    Should Be Equal As Numbers    ${product}    875

    # Convert string to uppercase using Python
    ${uppercase}=    Evaluate    $TEXT.upper()
    Should Be Equal    ${uppercase}    ROBOT FRAMEWORK AUTOMATION
```

```
# Get string length
${length}=    Get Length    ${TEXT}
Should Be Equal As Integers    ${length}    26

Log To Console    Sum: ${result}
Log To Console    Product: ${product}
Log To Console    Uppercase: ${uppercase}
Log To Console    Length: ${length}

Sleep    5s
```

MathLibrary.py

```
class MathLibrary:
    """Custom library containing mathematical keywords."""

    def calculate_sum(self, a, b):
        """Return the sum of two numbers."""
        return float(a) + float(b)
```

Output

3.1 Terminal Output

```
Terminal Local (2) x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment3> robot assignment3.robot
=====
Assignment3
=====
Custom Keyword And BuiltIn Operations      .....Sum: 60.0
Custom Keyword And BuiltIn Operations      .Product: 875
.Uppercase: ROBOT FRAMEWORK AUTOMATION
.Length: 26
Custom Keyword And BuiltIn Operations      | PASS |
-----
Assignment3                                | PASS |
1 test, 1 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment3\output.xml
Log:    C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment3\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment3\report.html
```

Result and Observation

The custom keyword returns 60 for 25 + 35. BuiltIn operations calculate 875, convert the phrase to uppercase, and return a length of 26; the assertions validate these values. Robot Framework normalizes Python method names into keyword names, so calculate_sum is called as Calculate Sum. The phrase contains 26 characters including spaces.

Conclusion

The experiment demonstrates the integration of Python-defined keywords with Robot Framework's BuiltIn operations and assertions.

Experiment 4: Assertions and Verification

Problem Statement

Perform a browser login and assert that the expected confirmation is displayed. In a separate test, send an API request and validate the HTTP status and selected JSON response fields.

Objective

To practice UI assertions and API verification using Robot Framework SeleniumLibrary, RequestsLibrary, and Should Be Equal As Strings.

Tools, Software, and Concepts Used

- Python 3, Robot Framework, SeleniumLibrary, RequestsLibrary, OperatingSystem
- Chrome browser and JSONPlaceholder REST API
- Assertions, HTTP GET, JSON parsing, page waits, screenshots

Implementation Steps

1. Import SeleniumLibrary, RequestsLibrary, and OperatingSystem.
2. Open the sample login page and enter valid credentials.
3. Submit the form and assert that the success message appears.
4. Create an API session and issue GET /posts/1.
5. Assert the status code, userId, id, and title, then log the response details.

Source Code —

assignment4.robot

```
*** Settings ***
Library    SeleniumLibrary
Library    RequestsLibrary
Library    OperatingSystem

*** Variables ***
${URL}      https://the-internet.herokuapp.com/login
${API_URL}   https://jsonplaceholder.typicode.com
${BROWSER}   Chrome
${SCREENSHOT_DIR}  screenshots

*** Test Cases ***
Verify Login Using Assertions
    [Documentation]  Verify login using Selenium assertions.
    Create Directory  ${SCREENSHOT_DIR}

    Open Browser  ${URL}  ${BROWSER}
    Maximize Browser Window

    # Wait for the login form to load
    Wait Until Element Is Visible  id=username  20s
    Wait Until Element Is Visible  id=password  20s
```

Log Login page loaded successfully.

Enter credentials

Input Text id=username tomsmith

Input Text id=password SuperSecretPassword!

Screenshot 1: filled login form

Capture Page Screenshot

... \${SCREENSHOT_DIR}/login_form.png

Submit login form

Click Button css=button[type='submit']

Wait for successful login confirmation

Wait Until Element Is Visible id=flash 20s

Wait Until Page Contains

... You logged into a secure area!

... 10s

Verify expected result

\${actual}= Get Text id=flash

Should Contain \${actual}

... You logged into a secure area!

Log Login assertion passed.

Screenshot 2: successful login

Capture Page Screenshot

... \${SCREENSHOT_DIR}/successful_login.png

Log To Console Login test completed successfully.

[Teardown] Close Browser

Verify API Response

[Documentation] Verify API status and response fields.

Create Session jsonplaceholder \${API_URL}

\${response}= GET On Session

... jsonplaceholder

... /posts/1

Verify HTTP status code

Should Be Equal As Integers

... \${response.status_code}

... 200

Verify response body

\${body}= Set Variable \${response.json()}

Should Be Equal As Strings \${body}[userId] 1

```
Should Be Equal As Strings  ${body}[id]  1
Should Be Equal As Strings
...  ${body}[title]
...  sunt aut facere repellat provident occaecati excepturi optio reprehenderit

Log To Console  API Status: ${response.status_code}
Log To Console  API Title: ${body}[title]
Log  API response verification passed.
```

Output

4.1 Login form populated

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

 Login

Powered by [Elemental Selenium](#)

Fork me on GitHub

4.2 Successful login

✔ You logged into a secure area!

Secure Area

Welcome to the Secure Area. When you are done click logout below.

Logout

Powered by [Elemental Selenium](#)

Fork me on GitHub

4.3 Terminal output

```
Terminal Local (2) x Local x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment4> robot assignment4.robot
=====
Assignment4
=====
Verify Login Using Assertions :: Verify login using Selenium asser...
DevTools listening on ws://127.0.0.1:64534/devtools/browser/b7a57b57-d91a-45f3-a2b4-eff85df45d9a
Verify Login Using Assertions :: Verify login using Selenium asser... Login test completed successfully.
Verify Login Using Assertions :: Verify login using Selenium asser... | PASS |
-----
Verify API Response :: Verify API status and response fields.
.C:\Users\Sristi\PycharmProjects\PythonProject\.venv\Lib\site-packages\urllib3
\connectionpool.py:1097: InsecureRequestWarning: Unverified HTTPS request is being made to host 'jsonplaceholder.typicode.com'. Adding certificate v
erification is strongly advised. See: https://urllib3.readthedocs.io/en/latest/advanced-usage.html#tls-warnings
warnings.warn(
.....API Status: 200
.API Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Verify API Response :: Verify API status and response fields. | PASS |
-----
Assignment4 | PASS |
2 tests, 2 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment4\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment4\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment4\report.html
```

Result and Observation

The browser test validates the secure-area confirmation. The API test checks HTTP 200 and the expected JSON fields. The suite is expected to report two passing tests when the browser and API are accessible.

OperatingSystem is imported for Create Directory. A test teardown closes the browser even when the login assertion fails. API requests require internet access; certificate or service diagnostics may appear without necessarily failing the test.

Conclusion

The experiment demonstrates assertion-based validation of both browser behavior and REST API responses.

Experiment 5: Test Setup and Teardown

Problem Statement

Configure a common test setup that opens the browser and logs in before each test case, and a test teardown that logs out and closes the browser after each case.

Objective

To demonstrate reusable test lifecycle keywords and consistent preconditions and cleanup for multiple test cases.

Tools, Software, and Concepts Used

- Python 3, Robot Framework, SeleniumLibrary, Chrome
- Test Setup and Test Teardown settings
- Reusable user keywords, conditional cleanup, browser lifecycle

Implementation Steps

1. Define the login URL and browser as variables.
2. Create the Login Before Each Test keyword to open Chrome, enter credentials, and confirm login.
3. Assign the keyword as the test setup.
4. Create Logout And Close Browser to check for the logout link, log out, and close Chrome.
5. Assign it as the test teardown and run both test cases.

Source Code —

assignment5.robot

```
*** Settings ***
Library    SeleniumLibrary
Test Setup    Login Before Each Test
Test Teardown    Logout And Close Browser

*** Variables ***
${URL}      https://the-internet.herokuapp.com/login
${BROWSER}  Chrome

*** Keywords ***
Login Before Each Test
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window

    Input Text    id=username    tomsmith
    Input Text    id=password    SuperSecretPassword!
    Click Button    css=button[type='submit']

    Wait Until Page Contains    You logged into a secure area!    10s

Logout And Close Browser
    ${logout_visible}=    Run Keyword And Return Status
    ...    Page Should Contain Element    css=a[href='/logout']
```

```
IF ${logout_visible}
  Click Link  css=a[href='/logout']
  Wait Until Page Contains  You logged out of the secure area!  10s
  Sleep  5s
END
```

Close Browser

*** Test Cases ***

Verify Secure Area

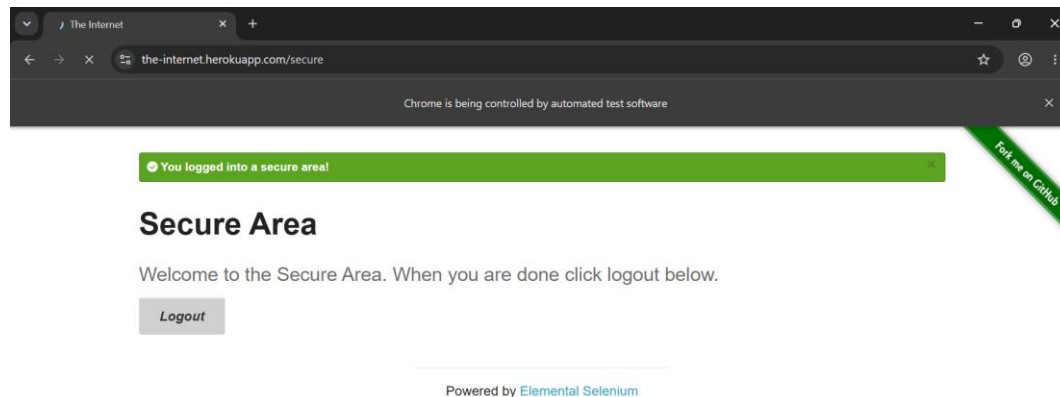
```
Page Should Contain  Secure Area
Sleep  5s
```

Verify Logout Link

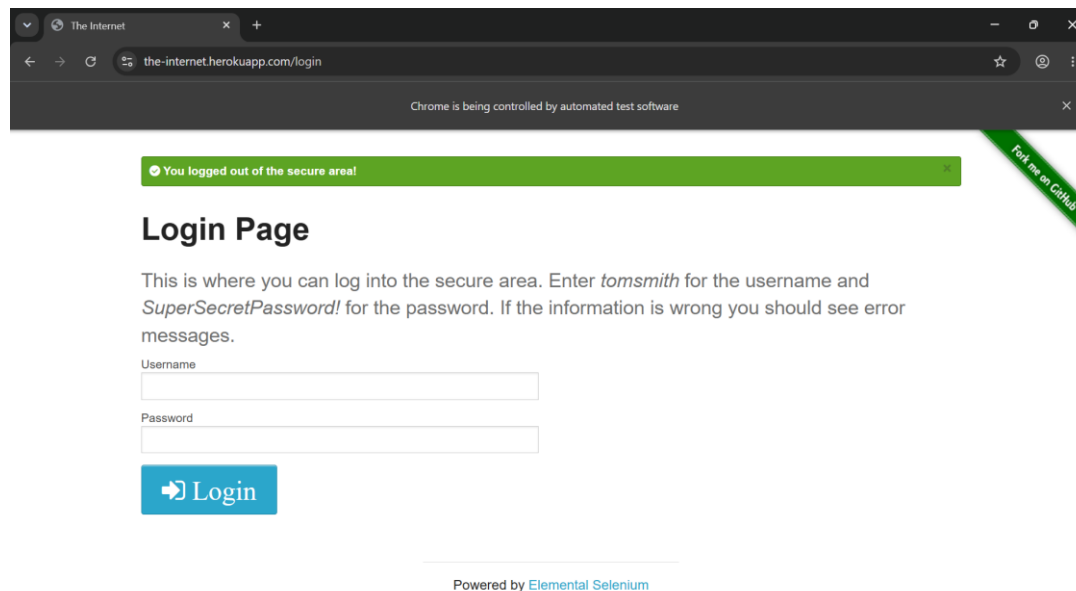
```
Page Should Contain Element  css=a[href='/logout']
Sleep  5s
```

Output

5.1 Logged-in secure area



5.2 Logout confirmation



5.3 Terminal execution summary

```
Terminal  Local x Local (2) x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment5> robot assignment5.robot
=====
Assignment5
=====
Verify Secure Area
DevTools listening on ws://127.0.0.1:61909/devtools/browser/f72fa612-196c-4ae7-a77c-c77a830974b0
..[9204:35648:0928/013154.013:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
.Created TensorFlow Lite XNNPACK delegate for CPU.
Verify Secure Area | PASS |
-----
Verify Logout Link
DevTools listening on ws://127.0.0.1:58977/devtools/browser/390713f7-1f8c-447f-a85c-b9c62677e7c6
..[13708:38528:0928/013217.845:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Logout Link | PASS |
-----
Assignment5 | PASS |
2 tests, 2 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment5\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment5\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment5\report.html
```

Result and Observation

The two test cases share the same login setup and cleanup. Each begins in the authenticated secure area and ends with logout and browser closure, provided the test reaches teardown. The teardown uses Run Keyword And Return Status to check whether the logout link exists before attempting to click it. The short pause allows screenshot capture of the logout confirmation.

Conclusion

The experiment demonstrates reusable setup and teardown, reducing duplicated steps and ensuring consistent test initialization and cleanup.

Experiment 6: Tags and Test Execution

Problem Statement

Assign tags to test cases and the test suite, execute selected or excluded tests using command-line options, and demonstrate parallel test execution.

Objective

To organize test cases with tags, control test selection using `--include` and `--exclude`, and run tests concurrently with Pabot processes.

Tools, Software, and Concepts Used

- Python 3, Robot Framework, SeleniumLibrary, Pabot
- Chrome and the-internet.herokuapp.com demo pages
- Suite and test tags, include/exclude selection, parallel execution

Implementation Steps

1. Create four test cases for the home, login, checkboxes, and dropdown pages.
2. Assign smoke, regression, login, checkbox, and dropdown tags as appropriate.
3. Add suite-level tags and common browser setup/teardown.
4. Run the full suite and selective tag commands.
5. Run the suite with Pabot using two processes and capture the outputs.

Source Code —

assignment6.robot

```
*** Settings ***
Library    SeleniumLibrary
Test Setup    Open Test Browser
Test Teardown    Close Browser

*** Variables ***
${URL}      https://the-internet.herokuapp.com
${BROWSER}  Chrome

*** Keywords ***
Open Test Browser
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window

*** Test Cases ***
Verify Home Page
    [Tags]    smoke    regression
    Go To    ${URL}
    Page Should Contain    Welcome to the-internet
    Sleep    5s

Verify Login Page
```

```
[Tags] smoke login
Go To ${URL}/login
Page Should Contain Element id=username
Page Should Contain Element id=password
Sleep 5s
```

Verify Checkboxes Page

```
[Tags] regression checkbox
Go To ${URL}/checkboxes
Page Should Contain Element css=input[type='checkbox']
Sleep 5s
```

Verify Dropdown Page

```
[Tags] regression dropdown
Go To ${URL}/dropdown
Page Should Contain Element id=dropdown
Sleep 5s
```

Output

6.1 Full suite execution

```
Terminal Local x + v
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6> robot assignment6.robot
=====
Assignment6
=====
Verify Home Page
DevTools listening on ws://127.0.0.1:51493/devtools/browser/45bd05c2-f439-4ca9-a773-6331bb19b4df
...[26044:29828:0928/013423.553:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Home Page | PASS |
-----
Verify Login Page
DevTools listening on ws://127.0.0.1:58469/devtools/browser/e56f1b49-fdc8-4ab6-a375-b339f6b2e11e
....[39100:11320:0928/013436.469:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_E
RROR
[39100:11320:0928/013436.470:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
[39100:11320:0928/013436.470:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
Verify Login Page | PASS |
-----
Verify Checkboxes Page
DevTools listening on ws://127.0.0.1:56129/devtools/browser/0f26e9fa-b91d-48df-bda9-526d075327ab
...[27988:29196:0928/013450.915:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Checkboxes Page | PASS |
-----
Verify Dropdown Page
DevTools listening on ws://127.0.0.1:54408/devtools/browser/bca8655b-c292-4679-9aca-4c6aee9cac9a
...[18068:1316:0928/013512.674:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Dropdown Page | PASS |
-----
Assignment6 | PASS |
=====
4 tests, 4 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\report.html
```

6.2 Smoke-only execution

```
(.venv) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6> robot --include smoke assignment6.robot
=====
Assignment6
=====
Verify Home Page
DevTools listening on ws://127.0.0.1:51734/devtools/browser/2b1cd1d8-2629-41b7-9437-a207175a06ef
...[40744:40432:0928/013731.315:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
[40744:39328:0928/013732.760:ERROR:chrome\updater\ipc\update_service_dialer_win.cc:75] Failed to open named pipe server process 20732: Access is den
ied. (0x5)
Verify Home Page | PASS |
-----
Verify Login Page
DevTools listening on ws://127.0.0.1:63242/devtools/browser/910ad5fb-2140-49a1-8def-9ec893cb0041
....[11788:23248:0928/013755.251:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Login Page | PASS |
-----
Assignment6 | PASS |
=====
2 tests, 2 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\report.html
```

6.3 Regression or exclusion run

```
(.env) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6> robot --exclude login assignment6.robot
=====
Assignment6
=====
Verify Home Page
DevTools listening on ws://127.0.0.1:64896/devtools/browser/1c4df4d3-6a0e-40bd-aec1-efe91eca6a4d
...[28592:39844:0928/014036.545:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Home Page | PASS |
-----
Verify Checkboxes Page
DevTools listening on ws://127.0.0.1:52380/devtools/browser/15ce6125-1090-4f09-ac37-930b67b4b031
...[32420:37176:0928/014052.784:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
Verify Checkboxes Page | PASS |
-----
Verify Dropdown Page
DevTools listening on ws://127.0.0.1:60909/devtools/browser/170984f1-6602-4060-98d5-b02851dc5e88
...[16544:12336:0928/014113.983:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ER
ROR
[16544:12336:0928/014113.990:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: PHONE_REGISTRATION_ERROR
Verify Dropdown Page | PASS |
-----
Assignment6 | PASS |
3 tests, 3 passed, 0 failed
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\report.html
```

6.4 Parallel execution

```
(.env) PS C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6> pabot --processes 2 assignment6.robot
Initialized logging in .\pabot_results
Robot Framework remote server at 127.0.0.1:8270 started.
Storing .pabotsuitenames file
2026-09-28 01:42:04.700263 [PID:18228] [0] [ID:0] EXECUTING Assignment6

DevTools listening on ws://127.0.0.1:58169/devtools/browser/2176f9c8-1c9c-4eaf-b044-9cfdfe43eeb
2026-09-28 01:42:55.223500 [PID:18228] [0] [ID:0] still running Assignment6 after 50.0s
[35776:35408:0928/014302.329:ERROR:google_apis\gcm\engine\registration_request.cc:291] Registration response error message: DEPRECATED_ENDPOINT
2026-09-28 01:43:07.106801 [PID:18228] [0] [ID:0] PASSED Assignment6 in 62.5 seconds
4 tests, 4 passed, 0 failed, 0 skipped.
=====
Output: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\output.xml
Log: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\log.html
Report: C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Module 4\Assignment6\report.html

Finalizing Pabot execution...
Stopping PabotLib process
Robot Framework remote server at 127.0.0.1:8270 stopped.
PabotLib process stopped
Total testing: 1 minute 2.50 seconds
Elapsed time: 1 minute 3.79 seconds
Logs flushed successfully.
```

Result and Observation

The full suite contains four tests; smoke selection contains two tests, regression selection contains three, and excluding the login tag leaves three. Pabot runs the suite using two worker processes; record the actual status from the terminal. The home-page test is tagged as both smoke and regression, so those subsets overlap. Use the Pabot command rather than passing --processes to the standard robot command. Separate runs are useful evidence for each execution feature, but should follow the instructor's stated screenshot requirements.

Conclusion

The experiment demonstrates tag-based organization, selective test execution, and parallel execution using Pabot.

Experiment 7: Reports and Logs

Problem Statement

Execute a Robot Framework suite that produces an HTML report and detailed execution log, and insert custom log messages at important points for debugging.

Objective

To generate Robot Framework output artifacts and use Log messages to trace test progress and verify execution.

Tools, Software, and Concepts Used

- Python 3, Robot Framework, SeleniumLibrary, Chrome
- Robot Framework HTML report and log generation
- Log keyword, output directory, output.xml, log.html, report.html

Implementation Steps

1. Create assignment7.robot with browser test cases and custom Log statements.
2. Run the suite with --outputdir results to collect the generated artifacts in one folder.
3. Open report.html to review the overall test summary.
4. Open log.html to inspect keyword-level execution and custom messages.

Source Code —

assignment7.robot

***** Settings *****

Library SeleniumLibrary

***** Variables *****

\${URL} https://the-internet.herokuapp.com/login

\${BROWSER} Chrome

***** Test Cases *****

Login Test With Detailed Logs

Log Test execution started. INFO

Open Browser *\${URL}* *\${BROWSER}*

Maximize Browser Window

Log Browser opened successfully. INFO

Verify login form elements

Page Should Contain Element id=username

Page Should Contain Element id=password

Log Login form elements verified. INFO

Enter credentials

Input Text id=username tomsmith

Input Text id=password SuperSecretPassword!

Log Username and password entered. INFO

Sleep 5s

Submit login form

Click Button css=button[type='submit']

Log Login form submitted. INFO

Verify successful login

Wait Until Page Contains You logged into a secure area! 10s

Log Login successful. INFO

Sleep 10s

Log final result

Log Test completed successfully. INFO

Close Browser

Log Browser closed. INFO

Verify Page Title

Open Browser \${URL} \${BROWSER}

Maximize Browser Window

\${title}= Get Title

Log Current page title: \${title} INFO

Should Be Equal \${title} The Internet

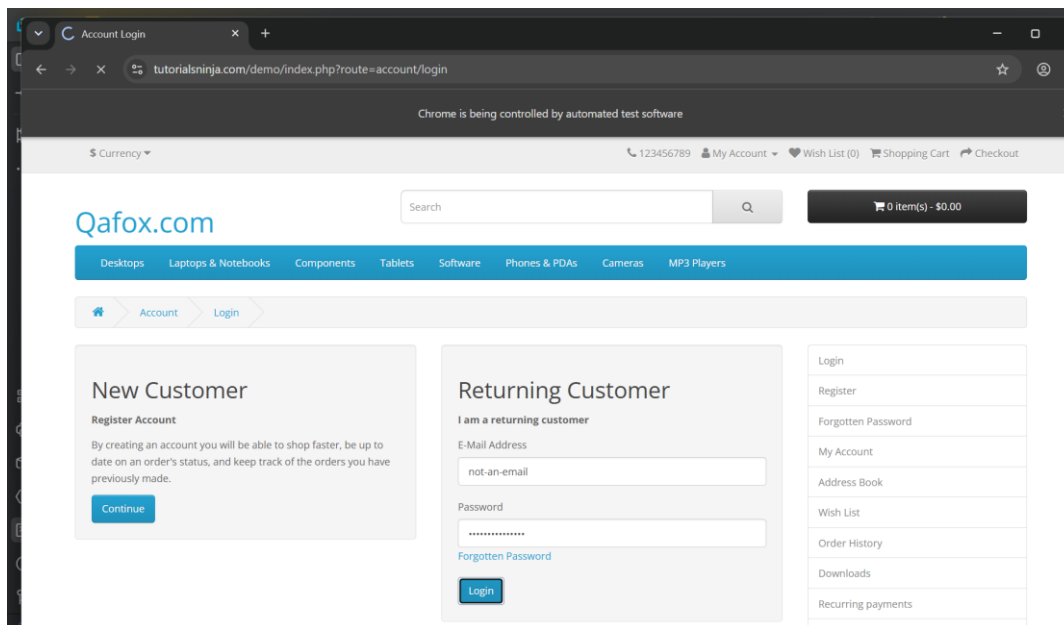
Sleep 5s

Close Browser

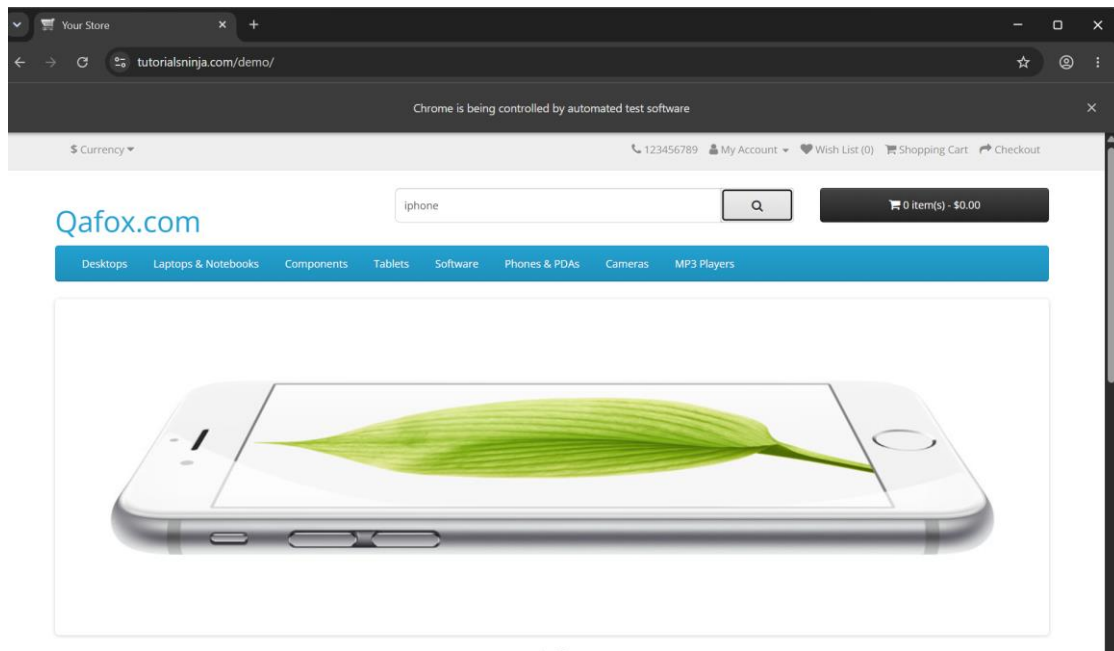
Log Page title verification completed. INFO

Output

7.1 Website



7.2 Browser login



7.3 HTML Report

pytest-report.html

Report generated on 28-Sep-2026 at 02:07:43 by pytest-html v4.2.0

Environment

Python	3.12.4
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.6.0
Plugins	• anyio: 4.10.0 • html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-25

Summary

4 tests took 00:00:34.

(Un)check the boxes to filter the results.

☒ 0 Failed, ☒ 4 Passed, ☒ 0 Skipped, ☒ 0 Expected failures, ☒ 0 Unexpected passes, ☒ 0 Errors, ☒ 0 Reruns ☒ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/pytest/test_login.py::test_invalid_login_shows_validation_message[row0]	00:00:08	
Passed	tests/pytest/test_login.py::test_invalid_login_shows_validation_message[row1]	00:00:10	
Passed	tests/pytest/test_product_search.py::test_product_search_returns_matching_results[row0]	00:00:08	
Passed	tests/pytest/test_product_search.py::test_product_search_returns_matching_results[row1]	00:00:08	

Result and Observation

The suite is expected to execute two tests and generate output.xml, log.html, and report.html in the results folder. The actual pass/fail count should be taken from the run. Custom Log statements mark browser startup, form validation, credential entry, submission, successful login, and test completion. The HTML log provides keyword-level details, while the report summarizes suite outcomes.

Conclusion

The experiment demonstrates Robot Framework reporting and diagnostic logging, providing evidence for test outcomes and a trace of execution.

Overall Module Result

The seven experiments cover the core Robot Framework automation workflow: introductory browser automation, variable-based and data-driven testing, custom Python libraries, UI and API assertions, test lifecycle management, tag-based and parallel execution, and report generation with logging. Record the actual execution results for each experiment after completing the practical work.

Overall Observation

- Robot Framework provides a readable keyword-driven structure for automated tests.
- SeleniumLibrary enables browser navigation, form interactions, element checks, and screenshots.
- Variables, loops, and external CSV data support reusable data-driven tests.
- Custom Python libraries extend the available Robot Framework keywords.
- Assertions verify browser outcomes and API response data.
- Test setup and teardown provide consistent initialization and cleanup.
- Tags and Pabot support selective and parallel test execution.
- HTML reports, XML output, logs, and screenshots provide evidence of execution.

Overall Conclusion

The module provides practical experience in designing and executing maintainable automation tests with Robot Framework. The experiments combine browser and API automation with reusable keywords, external test data, assertions, lifecycle management, execution control, and reporting. Together, they establish a foundation for structured and verifiable test automation.